



Industrially oriented voice control system

Adam Rogowski *

Faculty of Production Engineering, Warsaw University of Technology, ul. Narbutta 86, 02-524 Warszawa, Poland

ARTICLE INFO

Article history:

Received 2 July 2011

Received in revised form

17 August 2011

Accepted 25 September 2011

Available online 24 October 2011

Keywords:

Voice control

Speech recognition

Robotized systems

Manufacturing cells

ABSTRACT

The purpose of my research was to develop a novel voice control system for the use in the robotized manufacturing cells as well as to create tools providing its simple integration into manufacturing. A comprehensive study of existing problems and their possible solutions has been performed. Unlike some other works, it focused on the specific requirements that should be fulfilled by industrially oriented voice control systems. Analysis of existing solutions related to the natural language processing and those related to various voice control applications has been performed. Its goal was to establish the optimal method of voice command analysis for industrially oriented systems. Finally, a voice control system for manufacturing cells has been developed, implemented and practically verified in the laboratory. Unlike many other solutions, it takes into consideration almost all aspects of voice command processing (speech recognition, syntactic and semantic analysis and spontaneous speech effects) and – most importantly – their mutual influence. To provide the simple system customization (integration into any particular manufacturing cells), a special format for quasi-natural sublanguage syntax definition has been developed. A novel algorithm for semantic analysis, using specific features of voice commands used for controlling industrial devices and machines, has been incorporated into the system. Successful implementation in the educational robotized machining cell shows that industrial applications should be possible in the very next future.

© 2011 Elsevier Ltd. All rights reserved.

1. Introduction

The current paper presents research conducted by its author in the field of voice control of the robotized manufacturing cells with voice commands in quasi-natural language. The main objective was to develop a voice control system for the use in the robotized manufacturing cells as well as to create tools providing its simple integration into manufacturing.

The individual objects to be controlled by the voice commands may be very simple (like chucks) as well as very complex (like industrial robots). This fact influences the structure and complexity of voice commands used for their control. A comprehensive analysis of existing solutions related to the speech recognition and natural language processing had to be performed in order to evaluate their applicability to the voice control of the robotized manufacturing cells. The aim of that analysis was to establish the optimal method of voice command analysis in industrially oriented voice control systems. The following question has been asked: why have the voice control systems not been applied in

the industry yet? What are the limitations of existing speech recognition algorithms? It was also very important to answer the question about specific requirements that should be fulfilled by voice control systems employed in the industry. The majority of existing speech recognition and natural language processing systems do not deal with controlling any industrial devices. The very common language processing tasks are e.g. machine translation or question answering [1]. As far as voice control is concerned, the industrially oriented solutions constitute rather a minority. The most popular objects controlled by voice are mobile robots, particularly assistive robots. Many solutions in that field have been presented in the literature [2–13]. Not all of them could be applied in the industry although some of their elements may be very useful.

The remainder of this article is organized as follows. [Section 2](#) presents general challenges related to the speech recognition and natural language processing. [Section 3](#) discusses the problems specific for industrial applications of voice control systems. The advantages and disadvantages of existing solutions related to the speech recognition and natural language processing as well as their potential applicability in the industrially oriented voice control systems are discussed in [Section 4](#). Conclusions regarding the optimal functional structure of industrially oriented voice control system are presented in [Section 5](#). [Section 6](#) presents VCD format and its application for description of grammar rules.

Abbreviations: CFG, context-free grammar; DMAP, direct memory access parsing; MOP, memory organization package; NLP, natural language processing; SR, speech recognition; VCD, voice command description format

* Tel.: +48222348357; fax: +48228490285.

E-mail address: arog@cim.pw.edu.pl

A novel algorithm for semantic and pragmatic analysis as well as the rules for meaning description in VCD are discussed in Section 7. Section 8 presents a sample implementation: the voice control system of an educational robotized cell. Experimental results are discussed in Section 9. Finally, conclusions and plans for the future are described in Section 10.

2. Challenges

Generally, there are two main aspects of voice command processing:

- Speech recognition (phoneme or word recognition).
- Language processing (parsing, semantic analysis, pragmatic analysis).

Each of these research domains has its own challenges, solved and unsolved problems; therefore they are often considered separately, by different groups of researchers. Sometimes they are treated as separate, independent steps in voice command processing. As it will be shown later in this paper, such approach is not justified and – except very simple, functionally limited solutions – cannot be used for development of the real voice control systems that could be implemented and practically verified.

As far as speech recognition algorithms are concerned, it must be clearly stated that contemporary SR engines are by far not as reliable as human ear. Therefore various techniques are used in order to increase their reliability. One of them is training the user's voice profile (in the case of speaker-dependent SR engines). It is questionable, however, whether speaker-dependent solutions would be proper for industrial applications.

Some researchers tried to compare various speech engines in respect of their performance. For example, Ayres and Nolan [14] provide test results showing that Microsoft SAPI is among the best products, characterized by a very low recognition time and a very high recognition accuracy. No wonder that Microsoft SAPI was used in the voice control systems by many researchers [8,15–18]. Even if a very high recognition accuracy has been reported (about 98%), there is still a problem that any misunderstanding may have important (sometimes even catastrophic) influence on the functioning of an industrial system. Moreover, according to my experiences, the high recognition accuracy rate claimed by some authors can be achieved only under some specific circumstances (e.g. the use of high quality microphones and absolute lack of noise). Besides, let us notice that the recognition rate of a particular speech engine often depends on the engine's setup. For example, Microsoft SAPI lets the user freely adjust pronunciation sensitivity and recognition response time. Depending on the current settings (the number of their possible values is theoretically infinite) the recognition accuracy may be various. Therefore such a factor like recognition rate for a given SR engine cannot be practically unambiguously specified.

In my opinion, intensive research in the area of human hearing must be performed yet in order to provide much better reliability of speech recognition engines. However, even if the SR engines had achieved the reliability level of human ear, the problem of misunderstanding would be still present. Let us notice that human understanding of spoken sentences employs a kind of feedback between semantic analysis and speech recognition. We usually hear only a part of what is being spoken. We often do not clearly hear some phonemes or even the whole syllables. Nevertheless we “guess” what has been spoken because we know which words can be expected in a given context. In other words, semantic understanding influences the phoneme recognition process. This fact must be taken into account when developing

a voice control system. If we had restricted our project to syntactic and semantic analysis of voice commands, assuming the recognized sequence of phonemes or words to be input to our system, our solution (even perfectly developed) would have been good for automatic understanding of typed-in commands only and not for the spoken commands.

Another set of problems in the area of speech recognition (although partially overlapping with the domain of language processing) arises due to the spontaneous speech effects. Some of those effects are as follows:

- Pauses in speaking due to the speaker's hesitation: sometimes they may be mixed up with the end of the utterance.
- Word repetitions and inserted words with no importance for sentence meaning.
- Omitted (or spoken unclearly) words significant for sentence meaning.

From the language processing point of view, the most important problem is the variety of possible command formulations that is characteristic for the natural language. This variety has several sources. One of them is existence of synonyms i.e. different words having the same meaning. Another one is diversity of syntactic structures. This is not so apparent in English but constitutes a big problem in highly inflected languages. The solution presented in the current paper has universal character (independent of the language) and takes this problem into consideration, as well.

3. Specific features of industrial voice control systems

As it has been mentioned before, the majority of existing speech recognition and natural language processing systems do not deal with controlling any industrial devices. Among voice control applications, those used for controlling industrial devices constitute rather a minority. The reason is obviously not the lack of interest and not a conviction that the voice communication is not a suitable way for the human–machine communication in the industrial environment. Many authors have already proven the usefulness of voice-based human–machine interfaces and I am not going to discuss this issue here. For example, Kulyukin [8] presents the practical and scientific arguments for voice-based interaction between man and machine. Pires [19] shows the importance of the voice communication in industrial robotic cells in the future scenario, where humans and robots safely share the workspace and actively cooperate. Hence the problem consists rather in the lack of reliable solutions that could be applied in the real industrial environment. To solve this problem, it is necessary to ask the question about the specific features of industrial voice control systems. The answer to this question will help us to decide which elements of the general, non-industrial solutions can be adapted and what has to be developed anew.

Let us notice that some requirements that must be fulfilled by industrial voice control systems are more restrictive than those related to the non-industrial solutions. The most important ones are the following:

- Correct recognition of all the words of the voice command is particularly important. One falsely recognized word can be a reason for a wrong (and even dangerous) action performed by machine.
- The language used in engineering is rich in expressions representing numerical values. Wrong recognition of words representing numbers will be sure to result in wrong actions.
- Reaction to the voice command must be usually instant.

On the other hand, some requirements, significant in other domains, are not so restrictive in industrial voice control applications. Here are some examples:

- As the users of those applications will be qualified machine operators, they can be expected to adapt themselves to some restrictions regarding the voice command structure. Therefore a quasi-natural language could be used instead of the natural language. Nevertheless the language processing should take into consideration both lexical and syntactic diversity of voice commands.
- Unlike the users of assistive robots (who are often non-professionals or even people with disabilities), the machine operators can be expected to keep some discipline in speaking e.g. to avoid repetitions and insertions.
- Although the number of possible command formulations may be extremely high, the number of corresponding qualitatively different actions to be performed by machine is usually quite low. As it will be shown later, this fact influences the way the voice commands grammar can be defined.

4. Analysis of existing solutions

As it has been mentioned before, there is a relatively small number of existing operational industrially oriented voice control applications. They are usually based on a very limited number of simple voice commands [17,20,21]. Relatively more researchers deal with the voice control of mobile and assistive robots [2–13].

As far as natural language processing algorithms are concerned, the popular domains employing them are machine translation and question answering.

In this section the existing voice control applications will be discussed. Besides, the non-industrial solutions related to the speech recognition and language processing will be analyzed in order to evaluate their usefulness for the voice control of the robotized manufacturing cells.

4.1. Speech recognition reliability

As it has been mentioned before, the human understanding of spoken sentences employs a kind of feedback between semantic analysis and speech recognition. Such mechanism has not been directly applied in any speech recognition systems yet, nevertheless it is possible to use the context in order to increase the speech recognition accuracy in the following ways:

- Restrict possible voice commands to a small set of strictly defined utterances.
- Use a formal grammar (e.g. CFG) to define possible commands.
- Apply statistical NLP methods (e.g. n-grams).

All these methods let the SR engine restrict the number of utterances that can be expected, hence the wrong hypotheses can be rejected and the recognition reliability increased. The first method offers the highest reliability because pattern matching can be applied to the whole-sentence signal [22]. Unfortunately, it does not take into consideration the diversity of sentence formulations. The second method uses grammar rules to create language model, hence it is much more flexible in this respect. The recognition accuracy can be increased because the SR engine “knows” at every stage of sentence recognition which words can be expected next (based on the already enunciated part of the sentence).

The basic idea of statistical methods is to train the language model using big corpora of sentences [23]. As they provide a

mechanism for estimating the next word in the sentence, the recognition reliability can be improved. However, as it was noticed by Rosenfeld [24], the classical methods based on n-grams “use very little knowledge of what language really is” and this fact limits the quality of statistical language models. Therefore many attempts have been made to solve this problem. Including syntactic [25] and/or semantic [26] information provides much better results.

All implemented (in the real or virtual environment) voice control systems known to the author of this paper use the first or the second method [5–8,11,13–18,20,21]. Why are there no implementations of statistical methods (so popular in machine translation and question answering domains)? Stiefelhagen et al. [5], who deal with commanding a mobile assistive robot, claim that n-gram language models are not suitable for small domains where only little domain dependent data is available for training. Industrially oriented voice control is surely such a domain. Let us also notice that the set of possible voice commands will vary from one industrial application to another one. Each of them would require a separate training corpus that would have to be provided, hence the development of new applications could be troublesome. Therefore we may assume that statistical methods are not suitable for industrially oriented voice control systems. On the other hand, the grammar-based methods do not provide natural language processing. According to my knowledge, no one author of implemented voice control systems has fully overcome that problem.

Another way to improve the speech recognition is to base on the past user commands [27] or clarifications [22] and thus predict the most probable commands under given circumstances. Those methods can be used also in the industrially oriented voice control systems.

4.2. Spontaneous speech

Some spontaneous speech effects are usually not the area of interest of researchers who deal with NLP. It is often assumed that the sentences to be analyzed will be typed-in hence there will be no need to deal with speech recognition at all. As far as voice control applications are concerned, there is only a handful of them that try to cope with this problem. For example, Wasfy et al. [16] introduced the so-called “execution words” in order to provide an instant reaction to the voice commands despite possible pauses in speaking.

I know no existing (implemented) voice control applications that let the operator break (cancel) currently spoken command in order to speak it anew (except a rather primitive method, based on the length of silence period, that has been applied by Wasfy et al. [16]).

As far as inserted and/or omitted words are concerned, the best solutions of that problem are provided by statistical methods used for NLP [28,29]. However, using statistical methods for command recognition in industrial voice control systems is questionable for the reasons listed in the previous section. Besides, as it has been remarked in Section 3, the machine operators can be expected to keep some discipline in speaking hence there is no absolute necessity for taking into account that kind of spontaneous speech effects. The following considerations argue in favor of the same conclusion, too:

- Ignoring unrecognized words is not acceptable in industrially oriented voice control systems because those words may radically change the meaning of the whole command. Therefore inserted words (i.e. the words that are of no importance for sentence meaning) would be acceptable only if they could be recognized and understood by the system.
- Even one falsely recognized word can be – under some circumstances – a reason for a false action undertaken by machine.

Therefore “guessing” omitted (or unclearly spoken) words generally should not take place.

In contrast, such speech events like e.g. hesitations should be necessarily taken into consideration. Stiefelhagen et al. [5] who use CFG for language modeling show how to cope with hesitations and non-human noises. Those effects may occur at any time in the speech query hence they cannot be modeled manually in the grammar. Therefore acoustic models of the so-called filler words are used for this purpose.

4.3. Parsing and semantic analysis

Depending on the particular application, various algorithms for command analysis and understanding can be used. Semantic analysis is usually based on results of previously performed parsing (syntactic analysis). Therefore those two analyses cannot be treated separately.

The simplest way to transform the voice commands into the actions performed by machine is simple mapping of the utterances into execution code to be run in the machine's control system. Such solution does not require any algorithm for semantic analysis. Some voice control applications, particularly industrially oriented ones, use that scheme [14,17]. Although very simple and easy to implement (because pattern matching instead of pattern recognition is used), such solutions have substantial disadvantage: they do not take into consideration the variety of possible voice command formulations that is characteristic for the natural language. Potential users must memorize all possible commands. The only advantage is the easy way to ensure the instant reaction to the commands because there is no problem with recognition whether the command has ended or whether the user has made a pause.

From the industrial point of view it is important that not an efficient mechanism for defining commands containing numerical values can be provided. Separate patterns for commands including every possible value must be created. Therefore this method can be used for controlling very simple industrial devices only. There should be a very restricted number of possible actions to be performed by those devices. The execution procedures describing those actions should not take any (numerical) parameters.

A different way of command analysis is required for commands defined on the basis of a formal grammar (e.g. CFG). Many researchers dealing with the voice control (and particularly authors of implemented and practically verified applications) have chosen this way. Language definition using CFG provides much more flexibility. The variety of possible command formulations can be taken into account. However, this possibility has been used in part only. Kulyukin who developed a very robust (and practically verified) system for voice control of mobile assistive robots [8] uses CFG for description of various actions and various objects but he does not mention the use of synonyms in his paper. Wasfy et al. who developed a system for remote voice control of the simulated wind tunnel [16] introduced the list of synonyms in order to allow for issuing commands in more natural way (although their system is rather primitive as far as speech recognition is concerned e.g. it requires the user to separate the spoken words with short pauses). Command formulation diversity in the language defined by a formal grammar is also the subject of the paper by Lepratti [30] who deals with the so-called “linguistic ontology”. This paper has rather theoretical character. Oka et al. who developed a system for voice control of humanoid robots [6] use CFG for defining commands that show some diversity as far as both vocabulary and syntax are concerned. However, this paper presents a particular application only. No general rules have been developed nor presented.

The use of formal grammar constitutes substantial advantage for industrially oriented voice control systems: it provides a generic mechanism for defining expressions representing numerical values as exchangeable components of commands. Again, this possibility does not seem to have been fully recognized and employed by researchers. Only few researchers tried to cope with this problem e.g. Veiga et al. [20] who report creation of grammar rules representing 100 numbers (0–99) in their voice control system used in a robotic cell. However, their approach consists in laborious manual creation of grammar rules. As it will be shown later in this article, the use of CFG can provide more effective solution.

Analysis of commands expressed in the language defined by a formal grammar usually applies the so-called parse tree that can be easily created on the basis of grammar rules. The parse tree constitutes a base for semantic analysis. Various methods of meaning description are used. Among them, the use of semantic networks (in various forms) is very popular one [5,8]. Another method is the use of models based on logic e.g. predicate calculus. Application of L_{md} language, described by Yokota et al. [2], is an example thereof.

To avoid restrictions resulting from the language models based on formal grammar, the algorithms for analysis of commands in natural language can be applied. In the recent time, probabilistic and data-driven models have become quite standard throughout NLP [1]. Semantic analysis is performed using the so-called semantic labels, attached to the nodes of the parse tree [26,29,31]. The parse tree is usually obtained as a result of a shallow parsing. Those methods show excellent results in such domains like machine translation and question answering. However, due to the restrictions discussed previously, they are usually not used in voice control applications. I could find no example of their successful implementation in industrially oriented voice control systems. The work of Ge and Mooney [31] is one of probably very few attempts to apply statistical parser to analysis of some kind of commands used for controlling objects (actually those “commands” are rather instructions issued by a coach to the virtual soccer players). Successful implementation of that method was possible mainly due to the fact that Ge and Mooney did not content themselves with shallow parsing but created the so-called “semantically augmented parse tree” in order to translate it into description in a formal meaning representation language. However, most likely their solution would not be proper for industrial applications where very high command recognition reliability has the highest priority.

Another method of semantic analysis in NLP is based on the theory called conceptual dependency invented already a half century ago. First conceptual dependency parser was described by Schank [32]. Its main feature was the lack of syntactic analysis. This solution, although very effective, would show some disadvantages in voice control systems. First of all, it could not be easily combined with any mechanism for speech recognition reliability improvement (that were discussed earlier in this paper). Therefore this solution is more appropriate for analysis of typed-in commands. As far as possible industrial applications are concerned, the main disadvantage would be the lack of an efficient way to perform the recognition of expressions representing numerical values: the world description based on conceptual dependency is rather qualitatively than quantitatively oriented. Besides, “semantic parsing” proposed by Schank ignores inserted (unexpected) and unrecognized words. For the reasons explained in previous sections it is not acceptable in the industrially oriented voice control systems.

Nevertheless some applications (although not industrially oriented ones) using that method have been reported. More than a decade ago, Savage-Carmona et al. [13] developed Virbot—an assistive virtual robot controlled with the voice commands. The

commands were analyzed using conceptual dependency theory. Exactly the same approach has been much later coupled with speech recognition system based on SAPI [15]. As far as SAPI is concerned, the authors point out the usefulness of grammars specified using XML notation. How they can reasonably combine CFG-based speech recognition with semantic analysis based on conceptual dependency remains, however, unclear.

Another work has been presented by Yokota et al. [2]. Although there is no direct reference to the conceptual dependency theory in their paper, a careful reader may notice that they must have used elements of this theory for translating natural language expressions into the so-called conceptual structure. As far as voice recognition is concerned, the authors do not deal with this subject at all.

As opposed to the statistical methods, the solutions based on conceptual dependency theory do not need any training corpora. Another approach, combining statistical models with dependency grammars (not necessarily based on conceptual dependency), postulated by Rosenfeld [24], may be a promising research direction (although rather not for industrially oriented voice control systems).

5. Basic concept of industrially oriented voice control system

Analysis performed in preceding sections shows that the voice control based on NLP seems to be not appropriate solution in the industrially oriented applications. Similar conclusion has been drawn by Kulyukin [3] with regard to assistive robots domain. He states that speech recognition algorithms are not reliable enough to provide errorless robot control with voice commands in natural language. Sublanguage acquisition is proposed as an alternative. An experiment has been conducted in order to prove that subset languages are quickly learnable by people. Although it is desirable to use natural language commands in voice control applications, appropriateness of NLP solutions is restricted by many factors [4]. Therefore the systems based on formal grammar seem to be more promising. Research conducted by Stiefelbogen et al. [5] shows that sentence error rate is better for grammar-based than for n-gram-based speech recognition. However, as stated by Lauria [22], the main consideration for grammar-based approaches is generating a grammar with sufficiently wide coverage for the domain in question. Therefore the author of the current paper focused on the question how to take into consideration the great variety of possible command formulations using CFG. For that purpose, the special format for voice command description VCD has been created. It will be presented in the next section.

The industrially oriented voice control system developed by the author is based on the following assumptions:

- It should be easy to adapt it to any industrial robotized cells. Unlike most of other solutions described in the literature [5,6,8,9,17,20] it is not application oriented but it provides a generic tool for creating voice control applications.
- CFG-based language model should be used but diversity of possible command formulations must be taken into account.
- Requirements specific for industrially oriented systems (listed in one of the preceding sections) must be fulfilled.
- Spontaneous speech effects should be taken into consideration to some reasonable extent.

The system consists of three main modules:

- Speech recognition module (including vocabulary).
- Syntactic and semantic analysis module.
- Execution module (responsible for controlling machines and devices).

Because the system must be easy adaptable to any robotized cell, it fulfills the following conditions:

- Speech recognition module is universal (completely independent of particular application).
- Syntactic and semantic analysis module has generic character. There are fixed algorithms for parsing and for meaning extraction hence they do not have to be developed for each new application. However, sublanguages used for expressing commands in individual applications can be defined using textual VCD format. The same VCD format is used for command meaning description. This is the main difference between this system and the other ones known from the literature.
- Execution module is (of course) application oriented and must be developed for each application. However, a novel arithmetically oriented semantic analysis algorithm (that will be described later in this paper) simplifies the communication between (fixed) semantic analysis module and (application oriented) execution module.

The adaptability of this solution has been practically verified during laboratory exercises at the Warsaw University of Technology. The students without professional background in voice control were able to develop simple applications very quickly.

Requirements specific for industrially oriented voice control systems are also fulfilled. The use of regular grammar (CFG) restricts the number of utterances that can be expected, hence the wrong hypotheses can be rejected and the recognition reliability substantially increased. Numerical expressions being part of voice commands can be easily defined in VCD. Examples thereof will be presented later in this paper. The use of regular grammar has another important advantage. Although possible command formulations are numerous, they are unambiguously defined. Therefore in the most cases it is possible to recognize instantly the end of voice command without a need to apply any “execution words”. In this way the instant reaction to the commands can be ensured even if there are pauses (e.g. due to hesitation) in the operator's utterance. This is of a very big importance in the industrially oriented voice control systems.

Some other spontaneous speech effects have also been taken into consideration. The use of inserted words or phrases (i.e. the words or phrases with no importance for sentence meaning) is permissible on condition that those words or phrases are included in the VCD-based description as conditional elements. Ignoring unrecognized words or even phrases is also possible but not recommended.

There is possibility to break a mistakenly spoken command and speak it anew. Appropriate mechanism will be presented in the following section.

6. Sublanguage modeling using VCD

VCD format, developed by the author of the current paper, constitutes the core of the whole system. On one hand, it enables an easy way to define sublanguages used in individual applications. Sublanguages defined in this way may be characterized by a big variety of possible command formulations. On the other hand, the same VCD format is used for meaning description. Meaning description is usually application-specific but libraries of universal terms and concepts can be also created and then utilized in particular applications. Thus VCD format provides generic character of syntactic and semantic analysis module.

Let us discuss the grammar definition first. One of reasons for creating VCD format was the need to take into consideration a big diversity of command formulations characteristic for natural

language (including some spontaneous speech effects) while applying CFG for language modeling (to provide strict and unambiguous command understanding required in the industrially oriented voice control applications). In general, such combination would result in a huge number of necessary grammar rules. Fortunately, industrially oriented voice control systems are characterized by one important feature: although the number of possible command formulations may be extremely high, the number of corresponding qualitatively different actions to be possibly performed by machines is usually low. Of course, those actions can be described by many numerical parameters. The number of possible values of those parameters may be almost infinite. Nevertheless there are usually only few possible actions to be performed by machine. For example, a robot loading and unloading workpieces in/from machine tools may usually perform only 4 different actions: taking the workpiece from the pallet, placing it in the chuck, removing it from the machine tool and placing the workpiece on the pallet again. Each of those actions can be described by various parameters (e.g. specifying position on the pallet where the workpiece should be placed).

Considering this problem from the point of view of machine programming and control, we notice that the task of voice control system consists in transforming a voice command (that may be formulated in a huge number of ways) into a single procedure call. This procedure is one of only few possible procedures although it may take several numerical values as its actual parameters.

We can state that the number of possible voice commands is low although the number of their possible formulations is huge. Because of a very limited number of required top-level grammar rules, it is possible to define a CFG-based quasi-natural command language without an extreme effort.

Another problem is how to reduce the number of lower-level rules in order to provide a compact description of numerous possible command formulations. It must be noticed that each command can be seen as a sequence of phrases. Each phrase represents some information. Combination of information contained in component phrases determines the meaning of the whole command. In the following example we can distinguish three main phrases (every individual phrase is underlined):

Please place the workpiece in the second position on the left pallet

The first phrase determines the action to be performed. The second one determines the object. Finally, the third one determines destination where the object should be placed. A phrase can be seen as a sequence of lower-order phrases. For example, the third phrase in the above sentence can be divided in the following way:

in the second position on the left pallet

This process can be continued until the level of single words or basic phrases (the “leaves” of the parse tree) is reached. As a “basic phrase” we understand here the word sequence that expresses a basic information. It is application-dependent whether a particular phrase can be considered a “basic phrase”. For example “three-jaw chuck” would be a basic phrase if the only chuck that might be referred to in the command were the three-jaw chuck.

Many lower-order phrases may occur several times in different upper-order phrases. In some cases, particularly in the inflected languages, the lower-order phrases may occur in various sequences in the upper-order phrases. Therefore many components of different command formulations need to be defined only once. Similarly as programming languages can be compactly defined using Backus–Naur Form, the huge number of possible voice command formulations can be compactly described using VCD format.

An important factor that determines the diversity of sentence formulations in the natural language is existence of conditional

components. In the preceding sections they were referred to as “inserted words” (one of spontaneous speech effects). For example, the voice command can start with “grasp” or “please grasp”. The conditional word “please” could also occur at the end of that command. From the point of view of pragmatic analysis this word does not express any information at all. Nevertheless it must be taken into consideration in the parsing process.

The basic structure of VCD-based sublanguage definition for individual applications can be presented in the following way using extended Backus–Naur form (without detailed description of semantic data that will be subject of the next section):

```
sublanguage definition = set of statements;
set of statements = statement [set of statements , statement];
statement = command definition | phrase definition | escape expression | library reference;
command definition = command heading, set of rules;
phrase definition = phrase heading, set of rules;
command heading = '#com', [space] , command id, new line;
command id = name;
phrase heading = '#def', [space] , phrase id , new line;
phrase id = name;
set of rules = rule | set of rules , rule;
rule = phrase sequence, [semantic data], new line;
phrase sequence = phrase | phrase sequence , space , phrase;
phrase = word sequence | phrase reference | optional phrase reference | wildcard sequence;
word sequence = word | word sequence , space , word;
word = obligatory word | optional word;
obligatory word = alphabetic string;
optional word = '?', alphabetic string;
phrase reference = '*', phrase id, ['(', library name , ')'];
optional phrase reference = '?', phrase reference;
escape expression = '#esc', [space] , word sequence;
library reference = '#use', [space] , library file;
name = letter | name , letter | name , digit | name , '_';
alphabetic string = letter | alphabetic string , letter;
library file = ? filename ?;
space = ? space character ?;
new line = ? new line character ?;
wildcard sequence = '...';
digit = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9';
letter = ? alphabetic character ?;
```

An example of command description in VCD is presented below. Due to the lack of space it is a very simple one and does not properly reflect the existing potential of quasi-natural language modeling:

```
#com put_on_pallet
?please place the workpiece *destination
#def destination
in the *number position on the *side pallet
#def number
first
second
#def *side
left
right
#esc sorry once again
```

The main advantage of this format is its comprehensibility. The use of phrase references (phrase names preceded by asterisk e.g. “*destination”) as well as conditional phrases and words (preceded by question mark e.g. “?please”) allows to describe a

huge number of possible formulations in a very compact, yet transparent way.

Unlike conditional components that are unambiguously determined (and must be recognized), the so-called “wildcard sequences” denoted by “...” constitute the groups of words that can be simply ignored during recognition process. Due to the reasons mentioned earlier in this paper their use is not recommended in the industrially oriented applications.

It is worth mentioning that a language model created in VCD can be easily (automatically) converted into grammar rules used by speech recognition engines developed by others e.g. into XML-based syntactic rules used by Microsoft SAPI. Although SAPI has some disadvantages e.g. some restrictions on the use of recursive syntactic rules, I use SAPI for the Internet-based version of my system (providing voice control by remote users) because the remote users do not have to install any software (except freely available Microsoft SAPI) on their computers. In that case, the speech recognition is performed by SAPI on the client’s computer but semantic analysis is performed on our server. However, the remote voice control is a subject of another paper and I am not going to discuss it in detail now.

Please note that some symbols used in VCD (e.g. those denoting conditional phrases and “wildcards”) are similar to those used in SAPI (for the purpose of convenience for users who are already familiar with SAPI).

When the operator has spoken something mistakenly, he/she can wait for a longer period of time until the system assumes that the user has abandoned enunciating the command. This is rather inconvenient, though. Therefore the so-called escape sequence has been introduced. The escape sequence is a sequence of words that is responsible for interrupting currently spoken command. When the system encounters the escape sequence in the hypothesis (i.e. in the interim recognition result) then it interrupts the analysis instantly and starts a new recognition. In the above example, if the operator says e.g. “Place the workpiece in the first... sorry, once again!” then the command analysis is immediately abandoned and the correct command can be enunciated e.g. “Place the workpiece in the second position on the left pallet”.

The set of possible voice commands is usually application-specific hence the sublanguage definition has to be created for each application. Nevertheless a very big number of component phrases may have universal character i.e. they can be used in various voice commands (and in various applications). Phrases expressing numerical values are good examples thereof: they can be used in every application and they have always the same meaning. Therefore VCD format provides the use of libraries of phrases. Those libraries are separate textual files. Their structure can be presented in the following way using extended Backus–Naur form:

```
library of phrases = set of library statements;
set of library statements = library statement | set of library statements , library statement;
library statement = phrase definition | library reference;
```

“Phrase definitions” and “library reference” have been defined before. The number of various library references (“#use” statements) in the sublanguage model is unrestricted. Each library may also contain references to other libraries. VCD libraries can be created by various authors at various times. Therefore we cannot expect those authors to coordinate their work to ensure the unique names of phrases. To avoid ambiguity, references to “external” phrases must include the library filename in brackets e.g.

```
#use numbers
#def multiplication
*value(numbers) multiplied by *value(numbers)
```

where “value” is the name of the phrase defined in the library called “numbers”.

As mentioned before, the use of CFG increases recognition accuracy because the SR engine “knows” at every stage of sentence recognition which words can be expected next (based on the already spoken part of the sentence). The structure of VCD-based sublanguage model enables another way to improve the recognition accuracy. Namely, depending on the current situation, the system may easily “activate” and “deactivate” individual commands thus restricting substantially the number of expected utterances.

7. VCD-based semantic and pragmatic analysis

As it has been mentioned earlier, there are various approaches to the semantic analysis. As far as voice control applications are concerned, there is relatively little literature presenting detailed description of semantic analysis algorithms. Some of the authors present algorithms that are application-specific; some others do not present their algorithms at all. In my opinion, there is a lack of works aimed at creating semantic and pragmatic analysis algorithms that could be optimal for industrially oriented applications.

One of substantial assumptions behind the solution presented in the current paper is the use of a fixed algorithm for meaning extraction in all possible individual applications. Application-specific knowledge should be described using textual VCD format. In my opinion, it was better to provide a powerful and adaptable generic tool for meaning extraction than an algorithm for semantic analysis of the voice commands for a whole domain (e.g. robotics). The use of VCD format that integrates description of syntactic and semantic data is a novel approach. For example, Kulyukin and Steele [33] proposed automatic construction of grammar rules from semantic networks that are constructed manually [8]. VCD-based description (that can be easily created even by non-professionals) is used as a basis for automatic construction of both grammar rules and semantic network.

The following observations facilitated realization of this approach:

- Voice commands are usually imperative sentences. There is no distinct boundary between semantic and pragmatic analysis.
- As stated earlier in this paper, the final result of voice command analysis is a single procedure call. This procedure may take several numerical values as its actual parameters. In other words, the final result can be reduced to one symbolic and up to several numerical values. For example, a command formulated in the following way:
“place the workpiece in the second position on the left pallet”
may result in the following procedure call:
place (2,1)
where 2 is the position number and 1 is the pallet descriptor (“left”).
- Semantic analysis of voice commands can be described as arithmetically oriented one. Information contained in component phrases (represented by the nodes of the parse tree) has essentially arithmetic character. Even if that information is not explicitly arithmetic (e.g. “left”), it may be represented by numerical values (codes).
- Combining lower-order phrases into upper-order phrases usually results in data reduction. This phenomenon is particularly well recognizable when using expressions representing numerical values as a part of voice command. Let us shortly discuss it using an example. The individual words of the phrase “one hundred twenty five millimeters” represent five values: 1, 100, 20, 5 and 0.001 (the last one is a unit conversion

factor: its value has been established on assumption that the basic unit is one meter). When combining these words into phrases we get two values (125, 0.001) in the first step:

“one hundred twenty five millimeters”

and one value (0.125) in the second step:

“one hundred twenty five millimeters”

The basic mechanism for data reduction consists in performing arithmetical operations.

According to my knowledge, the above mentioned phenomenon (let us call it “data reduction principle”) is usually not utilized in the semantic analysis algorithms employed in voice control systems.

One of possible approaches to semantic analysis is presented by Ge and Mooney [31] who developed an algorithm consisting in translation of semantically augmented parse tree into a formal meaning representation. The final meaning representation is formulated in a meaning representation language (MRL) and it contains all (structured but practically unprocessed) information derived from individual words comprising the sentence. No data reduction principle is utilized at this stage of processing.

Another approach, presented by Kulyukin [8], consists in the use of the so-called memory organization packages (MOP) being the nodes of semantic network. This approach is particularly suitable for semantic analysis of CFG-based voice commands because – as proven by Kulyukin and Steele [33] – the input recognition capacity of CFGs is partially equivalent to the input recognition capacity of direct memory access parsing (DMAP) semantic networks. The so-called callbacks (procedures) are associated with MOPs. Based on token sequences (words) obtained from speech recognition, the MOPs are activated by a spreading activation algorithm based on DMAP [34]. Thus appropriate callbacks can be executed. However, robot voice control application presented by Kulyukin [8] seems to use the callbacks only at the top (root) level of semantic network in order to map the voice input into knowledge and control structures. Thus, as stated by the author, “robot’s dialog is based on language recognition instead of language understanding”.

Solution presented in the current paper utilizes execution of procedures that perform some arithmetic calculations at every level. Those procedures are associated with the nodes representing phrases or the entire command. Their task is to calculate numerical values (let us call them “node parameters”) representing information contained in the phrases. Such scenario requires node parameters (variables) to be associated with all nodes. Each node representing a voice command, a phrase or a single word is assigned one or several node parameters representing information contained in that command, phrase or word. Single words (represented by terminal nodes i.e. the “leaves” of semantic network) are assigned constant values only. For example, the word “five” is simply assigned the value 5.

Each non-terminal node (representing a phrase or the entire command) can be assigned a procedure that calculates the parameters of that node. This procedure can be usually reduced to a single arithmetic expression containing references to the parameters of children nodes representing component phrases or/and words. The parameters of children nodes are referenced as variables named p1, p2, p3, etc. where p1 is the parameter of the first child node (representing the first component phrase/word), p2 is the parameter of the second child node (representing the second component phrase/word) and so on. If the component phrase contains information represented by more than one parameter, the names of variables are indexed using values in square brackets e.g. p1 [1], p1 [2], etc.

The above mentioned arithmetic expressions are the part of VCD-based sublanguage definition. They are contained in “semantic data” referenced to in the previous section. Now, let us define “semantic data” using the extended Backus–Naur form:

```
semantic data = ':' , set of expressions;
set of expressions = expression | set of expressions , delimiter ,
expression;
delimiter = ':' | new line , ';' , new line;
expression = ? arithmetic expression representing constant value ?
| ? arithmetic expression containing variables p1,p2,p3... representing parameters ?
```

If the information contained in the phrase is represented by exactly one node parameter then the set of expressions consists of one expression only. Otherwise the first expression determines the value of the first node parameter, the second expression determines the value of the second node parameter and so on.

As an example, a very little part of VCD-based language definition containing semantic data is presented below. Due to the lack of space, only few variants of the phrase representing numerical values are shown:

```
#def distance
*numerical_value *units:p1*p2
#def numerical_value
ten:10
eleven:11
twelve:12
#def units
millimeters:0.001
centimeters:0.01
meters:1
```

After the user utters the command, the node parameter corresponding to the phrase named “distance” will be calculated as multiplication of the node parameters corresponding to the phrases named “numerical_value” (p1) and “units” (p2). The value of “distance” will represent the distance in meters. For example, when the user says “eleven centimeters” then this value will be calculated as $11 \times 0.01 = 0.11$.

The node parameters of the root node (corresponding to the whole command) can be directly used as parameters of the execution procedure.

As mentioned before, the simple arithmetic expressions usually suffice to calculate the node parameters. Therefore it is possible to include the semantic data in VCD-based language definition. However, more complex algorithms are needed in some cases. A good example of such case is the cooperation between voice control and vision systems. Data provided by the vision system may need complex processing in order to determine the value representing the information contained in the phrase analyzed by the voice control system (e.g. “the second workpiece from the left”). In this case the arithmetic expression contained in “semantic data” may call some external functions that are defined elsewhere (e.g. in dynamic link libraries).

It is worth mentioning that the application of VCD libraries (discussed in the previous section) let define phrases that may be common for various applications. Description of those phrases contains semantic data that should have universal meaning. Phrases expressing numerical values are excellent examples thereof. Because of the universal character of semantic data associated with “library phrases”, the solution presented in the current paper can be characterized as partially based on language understanding rather than on simple mapping between voice input and control structures.

8. Laboratory implementation

The system presented in this paper is a generic tool that allows to create industrially oriented voice control applications in a relatively simple way. In order to develop a new application, two following steps have to be taken:

- Define the command sublanguage (using VCD format).
- Create the application-oriented execution module (for controlling the particular cell).

Based on this system, several test applications have been already developed. The most advanced one is the system for controlling an educational robotized manufacturing cell in the Laboratory for Automated Manufacturing at the Warsaw University of Technology (<http://www.cim.pw.edu.pl/arog>). This application constituted a basis for practical verification of the method.

Fig. 1 presents the general view of the robotized cell. The cell consists of the following main components:

- milling machine (on the left side of the cell);
- lathe (on the right side);
- industrial robot;
- fixtures (machine vice on the milling machine and tailstock on the lathe);
- pneumatic slide.

There are two pallets for workpieces (one on the left side and the other one on the right side).

The role of pneumatic slide is to move the robot from the left side (where it has access to the milling machine and to the left pallet) to the right side (where it has access to the lathe and to the right pallet) and vice versa. Fig. 1 shows the robot in the left position.

All machines and devices are controlled by the cell's main computer via RS-232. The communication includes uploading control programs to the controllers of both machine tools and the robot. Several cameras used by the image recognition system are placed within and around the cell. One of them, providing the view of the right pallet, is visible on the right side of Fig. 1.

The voice control system runs on a separate computer and communicates with the cell's main computer over the network. The technical measures that had to be taken in order to adapt the manufacturing cell for remote control have been presented in another paper [35]. Due to the use of a wireless microphone the user has a full freedom to move around.

According to assumptions presented earlier in this paper, the execution module had to be developed especially for this application. It is based on a specially designed software (Arlang programming language). A program written in Arlang may call the procedures defined in the specialized library. Those procedures are responsible

for automatic generation of control programs for machines as well as for communication with the cell's main computer.

The basic procedures used for robot control are the following: LOAD (loading individual robot instructions), EXECUTE (running the sequence of instructions in the robot controller).

Some basic procedures used for controlling other machines and devices are as follows: LSEND (sending NC program to the lathe controller), MSEND (sending NC program to the controller of the milling machine), TOPEN, TCLOSE (opening or closing the tailstock), VOPEN, VCLOSE (opening or closing the machine vice), RLEFT and RRIGHT (moving the robot slide left or right). Another Arlang library contains procedures and functions responsible for communication with speech recognition modules. The whole system functions according to the general scheme presented in Fig. 2.

The basic procedures and functions used for communication with speech recognition modules are the following: SPEAKER, MICROPHONE, ACTIVATE, ACTIVATE ALL, COMMAND and PARAMETER. Procedure SPEAKER is responsible for text-to-speech processing. For user's convenience, different voices can be assigned to individual machines. Procedure MICROPHONE turns on the SR engine (in the current version it is Microsoft SAPI engine) in order to record operator's voice command. If not otherwise specified, all top level rules are active. However, in order to improve the speech recognition reliability, it is better to activate only those top level rules, which correspond to the commands that are expected in the current context. It is possible to do it programmatically using ACTIVATE procedure e.g. "ACTIVATE (put_on_pallet)" activates the top level rule corresponding to the command defined after "#com put_on_pallet" statement in VCD-based language definition. Procedure ACTIVATE may be called several times with different parameters. Procedure ACTIVATE ALL sets the initial state. The results of voice command recognition can be accessed using COMMAND and PARAMETER functions. Function COMMAND returns the command id of the recognized command (see Section 6). Function PARAMETER returns the values of the "node parameters" of semantic network's root node (see Section 7). Those parameters can be usually directly used in the instructions sent to the manufacturing equipment.

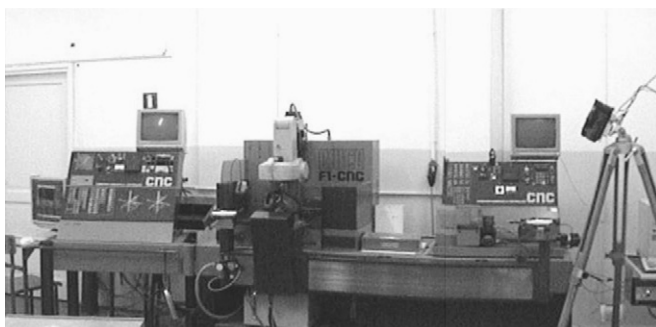


Fig. 1. General view of the robotized manufacturing cell.

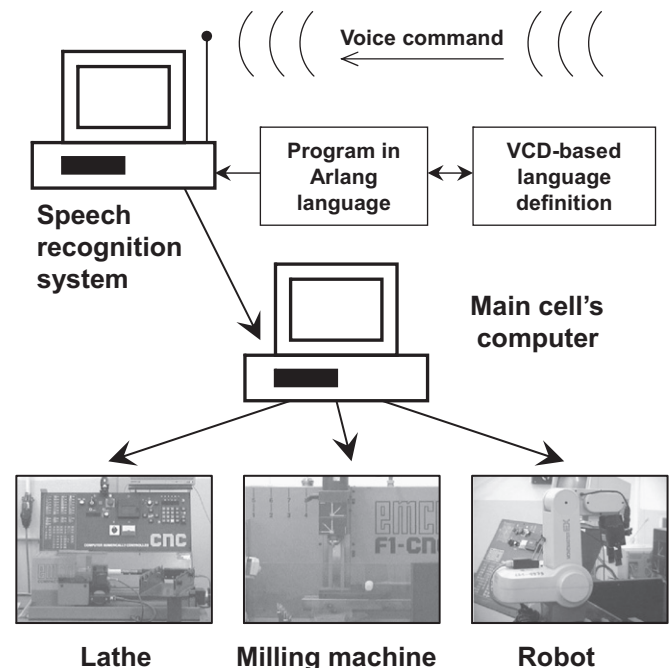


Fig. 2. Integration of the speech recognition system into manufacturing cell control.

There are two possible modes of cooperation between the SR engine and the running Arlang program. The first one consists in turning on the SR engine each time when MICROPHONE procedure is called. Program execution is then blocked until a valid command has been spoken by the operator.

The other mode consists in running SR engine and Arlang program in separate threads. Speech recognition can be performed independently of running Arlang program. When MICROPHONE procedure is called, the results of last speech recognition attempt are analyzed. This mode has several advantages. First, the voice communication does not block the program execution. Second, the operator can enunciate the command any time without waiting for a signal (e.g. automatically generated spoken statement) that speech recognition starts. Third, it provides the easier handling of operator's self-corrections. However, there is also a disadvantage. As the operator does not know the moment when the voice command is processed, he/she may unintentionally "cancel" the command with incidentally spoken utterance.

As discussed earlier in this paper, the syntactic and semantic analysis module uses a fixed algorithm. Nevertheless it can be applied for controlling different manufacturing cells because the "exchangeable" semantic data is contained in the VCD-based textual description. Appropriate sublanguage has been defined in order to control the manufacturing cell described in this section.

A video clip available at the web page <http://www.cim.pw.edu.pl/arog/glosowe/glosowe3.wmv> presents functioning of this application.

Depending on the objects (machines or devices) to be controlled, the structure and complexity of voice commands may be various. Sets of simple commands have been defined for controlling auxiliary devices e.g. "Close the vice", "Move robot left" and so on. Nevertheless the synonyms and various formulations have been taken into consideration. Therefore such commands like "Please close the vice" or "Close the fixture" can be used interchangeably. Ambiguities resulting from some formulations (e.g. the command "close the fixture" may refer to the vice in the milling machine or to the tailstock in the lathe) are resolved using context information. Information about previously undertaken actions can be used for this purpose. Some situations impose the use of clarification dialogs. In the case of critical actions the operator may be asked for the confirmation e.g. the question "Robot is about to move right. Do you confirm?" is asked in order to warn the operator that the robot will move into the space occupied by him/her.

The commands used for controlling machines are usually more complex sentences. Their possible formulations are very numerous. However, the sublanguage of those commands could be effectively modeled using VCD. In order to avoid the actions resulting from the false command recognition some measures had to be taken. When false recognition is highly possible (e.g. when similarly sounding words like "fifteen" and "fifty" may be expected) then the system enunciates what it has understood and what action is going to be undertaken ("Loading the program number fifty"). Thus the operator has time to cancel the command and speak it again if necessary. In order to simplify command formulations yet avoid ambiguities, the information derived from vision system can be used. For example, when the operator commands to put the workpiece on the pallet but does not indicate a specific position ("Robot, place the workpiece on the right pallet"), then the system uses the image recognition results in order to find a position that is not occupied by other workpieces.

An interesting aspect of this application is the use of voice commands for controlling machine tools. This is in contrary to the most of known voice control applications that are focused on robots. Actually, voice communication with machine tools has been the subject of interest for many years. More than a decade ago Schlick et al. [21] investigated the use of the head-mounted display equipped with speech-processing unit as an interface between the operator and CNC machine tool. Several years later, in the year 2006,

"American Machinist" reported about commercial version of such device ("Talk nicely to your machine", 04/20/2006). However, those solutions are used for auxiliary activities and not for controlling the machining process. Application presented in this paper uses voice communication for running CNC programs.

In order to provide a better explanation on functioning of this laboratory implementation, an (very little) excerpt from the VCD-based sublanguage definition is presented below:

```
...
#com yes
yes
#com no
no
#com robot_right
?please move robot right
...
```

The corresponding fragment of Arlang code is the following:

```
...
microphone
if command(robot_right)
speaker(Robot is about to move right. Do you confirm?)
activate(yes)
activate(no)
microphone
if command(yes)
rright
else
...
```

As mentioned before, the MICROPHONE statement is responsible for turning on the SR engine. The user's command is being recorded and analyzed. If the utterance corresponds to the one of possible formulations listed after *#com robot_right* statement in VCD-based language definition then *speaker* procedure is called. The system asks the user whether he/she confirms the action. The next *microphone* statement is preceded by two *activate* statements in order to restrict possible answers to "yes" or "no". If the user answers "yes" then the robot slide moves right.

9. Experimental results

Experimental verification of the method presented in the current paper has been performed on the basis of laboratory implementation described in the previous section. It must be emphasized that my research was aimed at developing a generic tool that allows to create industrially oriented voice control applications in relatively simple way. Therefore its results cannot be reduced to the application presented in Section 8. This application (although it is a fully operational and autonomous system) constitutes merely a basis for "sample" practical verification of the method. I would describe it as a kind of "case study".

Since the reliability of voice control system depends not only on the general method but also on the application-specific details, the experimental results presented in this section must be understood as related to this individual implementation. The reliability of other potential applications will strongly depend on many factors that are not known in advance e.g. on the set of possible commands, the proper use of top rule activation mechanism, clarification dialogs, confirmation requests and so on. Nevertheless, the performance of this implementation is an indication for the usefulness of the method.

The following setup was the basis for experiments:

- SAPI speech recognition engine was used.
- Standard SAPI pronunciation sensitivity was set to 50% (exactly in the middle between "low" and "high").

- Accuracy vs. recognition time factor was also set to 50%.
- Background adaptation was turned on (in order for the system to automatically adapt to the user's voice during regular use) although no intensive training was performed. Since SAPI allows to create several independent user profiles, I decided this option would be acceptable also in an industrially oriented voice control system that can be operated by various people.
- Since the system should work in the industrial environment, I did not want to create ideal conditions (characteristic for laboratory experiments). Therefore a very simple, low quality microphone was intentionally used for voice recording. Please note that Microsoft requires microphone to “be a high quality device with noise filters built in”.
- The number of possible (qualitatively different) actions to be launched by voice commands was 18. VCD-based sublanguage definition comprises 85 lines of code, which is equivalent to more than 16,000 of possible formulations of voice commands. The great number of those formulations results mainly from the fact that the numbers in the range 1–999 may constitute a part of command (namely as NC program identifiers).

In order to check the system reliability, the user commanded machines and devices of the robotized cell by voice. The experiment was conducted twice. Each time it lasted about 30 min and it was stopped after 100 correct commands had been uttered. This corresponds to about 6 min speech. The first time the user was distant from the cell hence no noise was involved. The second time the user was in close proximity of working equipment. Besides, some people were talking in the surrounding area. The following values were measured (see Table 1):

- The number of commands that have been correctly and immediately (i.e. without any need to repeat them) performed by machines and devices (column A).
- The number of correct commands that have been not understood by the speech recognition system (column B1).
- The number of incorrectly spoken commands that were either canceled by the user or rejected by the speech recognition system (column B2).
- The number of commands incorrectly recognized (misinterpreted) by the speech recognition system (column C).
- The number of misinterpreted commands that have been automatically rejected (column C1).
- The number of misinterpreted commands that have been canceled by the user (column C2).
- The number of failures (column D). System response to the voice command is considered failure when incorrect actions are taken by machines or devices.

In both cases there were no wrong actions taken by any device (see column D) hence the system as a whole can be described as reliable. However, the analysis of other values presented in Table 1 indicates potential weak points.

The number of commands that have been immediately and correctly recognized and performed (column A) reflects mainly the quality of voice input equipment and the reliability of SR engine (Microsoft SAPI). Taking into account the poor quality of the microphone (as explained earlier, it was intentional) and a

large number of possible formulations of commands (above 16,000) the results are quite good due to the use of CFG (92% of commands have been immediately and correctly performed). However, when the noise is involved, this value drops significantly. On the other hand, if we had used high quality microphone with noise filter built in, the results would be probably much better.

Although there was a relatively big number of correctly spoken commands that were not recognized by SR engine (particularly when the noise was involved), this fact does not influence the evaluation of system reliability because no wrong actions were taken. In such cases the user is informed by the system that his/her command could not be understood. This is, of course, not very comfortable because the user has to repeat his/her command. Nevertheless such situations cannot be considered failures.

The number of falsely spoken voice commands (column B2) reflects the mistakes made by the user and not by the system (these commands have been either canceled by the user or automatically rejected by the system).

Potential danger is reflected by the values contained in column C. These are the numbers of falsely interpreted commands or other utterances falsely interpreted as commands. Wrong interpretation may result in wrong actions taken by machines. As it can be seen in Table 1, this danger rises when the noise is involved. Two cases that occurred despite the lack of noise (see the first row, column C) were caused by existence of similarly sounding words in two different commands (e.g. “fifty”–“fifteen”). Such errors may occur not only in human–machine communication. They are quite common also in the communication between humans. Appropriate mechanisms must be provided in order to avoid the wrong actions. Sometimes the system may automatically reject the command that should not be performed in the current context (see column C1). Otherwise the user intervention may be necessary. When a mistake is highly possible, the system should either ask for confirmation or at least inform the user what it has understood (before the action is taken). In this way the user is provided with the possibility to break the command (see column C2).

Summarizing the experimental results, it can be stated that the main potential sources of a limited voice control system reliability are deficiency of speech recognition engines and the presence of noise. However, the influence of these factors can be neutralized by the application of appropriate mechanisms.

10. Discussion

The final result of research presented in this paper is the industrially oriented voice control system adaptable to any manufacturing cell. The rules that underlie this system have been elaborated. The most important of them are the following:

- The mutual influence of all stages and aspects of voice command processing must be taken into account.
- Sublanguage definition based on the regular grammar is an optimal solution under condition that the variety of voice command formulations is provided.
- Mechanisms for handling of some spontaneous speech effects must be included, particularly those responsible for operator's hesitations and self-corrections.
- Immediate reaction to operator's commands is required.
- Semantic and pragmatic analysis module must be easy to adapt to individual applications.

The core of my solution is VCD format that helps to fulfill the above requirements. The use of this textual format that integrates description of syntactic and semantic data is definitely a novel and unique solution. Even non-professionals are able to define the sublanguages (including their semantics) for particular applications.

Table 1
Test results.

	A	B1	B2	C	C1	C2	D
No noise	92	6	6	2	1	1	0
Noise involved	82	13	3	5	1	4	0

The use of “data reduction principle” together with application of semantic data in the form of arithmetic expressions in VCD-based sublanguage definition provides a convenient and transparent way to define the semantic analysis of phrases containing even very complex information. This solution seems to be unique within the voice control systems.

The successful implementation described in Section 8 and experimental results presented in the previous section indicate that industrial applications should be possible in the very next future. This application proved user-friendly due to the sublanguage definition providing great variety of possible command formulations (according to author's knowledge it is one of the most advanced industrially oriented voice control systems based on quasi-natural language). Despite that variety it shows a very good reliability. Although there were some cases of false command recognition, the mechanisms built in the system (e.g. clarification dialogs, confirmation requests) never failed to prevent the machines from undertaking wrong actions. Nevertheless it is too early to draw the generalized conclusions about reliability of the future industrial voice control systems. We must be aware that it will depend on individual applications, set of possible commands and mechanisms used for reliability improvement. However, the results achieved until now are very promising.

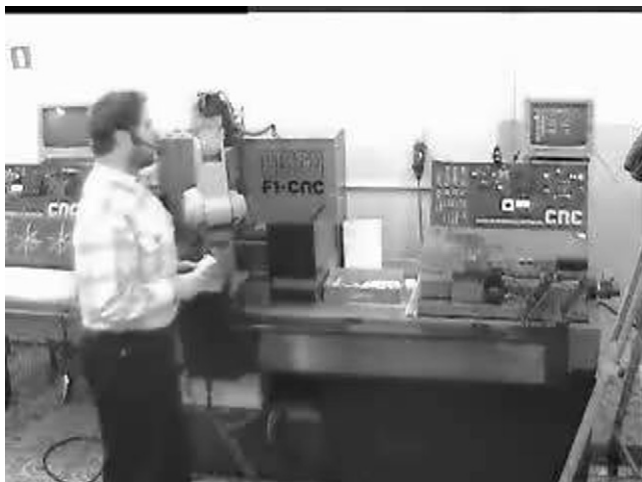
The future plans comprise the following:

- Development of universal VCD libraries of phrases specific for manufacturing and robotics domains.
- Further research on remote voice control.
- Research related to the use of voice commands for automatic NC code generating for machine tools. In the application described in the previous section the only method to control machine tools by voice was calling NC programs by their numbers. The NC programs had to be created earlier. An alternative approach could consist in automatic NC code generating based on verbal task description.

There is also a plan to create the webpage where all interested researchers could exchange their ideas regarding specialized VCD libraries.

Appendix A. Supplementary material

The following are the supplementary data to this article:
Video S1 presents functioning of this application.



Video S1. Video clip presenting voice control of the manufacturing cell. A video clip is available online. Supplementary data related to this article can be found online at [doi:10.1016/j.rcim.2011.09.010](https://doi.org/10.1016/j.rcim.2011.09.010).

References

- [1] Jurafsky D, Martin J. Speech and language processing. An introduction to natural language processing, computational linguistics, and speech recognition. Upper Saddle River, New Jersey: Prentice Hall; 2000.
- [2] Yokota M, Sugita K, Oka T. Natural language understanding based on mental image description language L_{md} and its application to language-centered robot manipulation. *Artificial Life and Robotics* 2008;13(1):84–8, [doi:10.1007/s10015-008-0564-y](https://doi.org/10.1007/s10015-008-0564-y).
- [3] Kulyukin V. Talk the walk: robotic NLP vs. human sublanguage acquisition. In: Proceedings of the AAAI spring symposium on multidisciplinary collaboration for socially assistive robotics, Palo Alto, CA; 2007. p. 33–5.
- [4] Kulyukin V. On natural language dialogue with assistive robots. In: Proceedings of the 1st ACM conference on human–robot interaction, Salt Lake City, Utah; 2006. p. 164–71. [doi:10.1145/1121241.1121270](https://doi.org/10.1145/1121241.1121270).
- [5] Stiefelhagen R, Fugen C, Gieselmann P, Holzapfel H, Nickel K, Waibel A. Natural human–robot interaction using speech, head pose and gestures. In: Proceedings of the IEEE-RSJ international conference on intelligent robots and systems, Sendai, Japan; 2004. p. 2422–7. [doi:10.1109/IROS.2004.1389771](https://doi.org/10.1109/IROS.2004.1389771).
- [6] Oka T, Abe T, Sugita K, Yokota M. RUNA: a multimodal command language for home robot users. *Artificial Life and Robotics* 2009;13(2):455–9, [doi:10.1007/s10015-008-0603-8](https://doi.org/10.1007/s10015-008-0603-8).
- [7] Bos J, Oka T. A spoken language interface with a mobile robot. *Artificial Life and Robotics* 2007;11(1):42–7, [doi:10.1007/s10015-006-0397-5](https://doi.org/10.1007/s10015-006-0397-5).
- [8] Kulyukin V. Human–robot interaction through gesture-free spoken dialogue. *Autonomous Robots* 2004;16:239–57, [doi:10.1023/B:AURO.0000025789.33843.6d](https://doi.org/10.1023/B:AURO.0000025789.33843.6d).
- [9] Bugmann G. Challenges in verbal instruction of domestic robots. In: Proceedings of the ACER'03 1st international workshop on advances in service robotics, Bardolino, Italy; 2003. p. 112–6.
- [10] Lauria S, Bugmann G, Kyriacou T, Klein E. Mobile robot programming using natural language. *Robotics and Autonomous Systems* 2002;38(3–4):171–81, [doi:10.1016/S0921-8890\(02\)00166-5](https://doi.org/10.1016/S0921-8890(02)00166-5).
- [11] Theobalt C, et al. Talking to Godot: dialogue with a mobile robot. Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems 2002:1338–43, [doi:10.1109/IRDS.2002.1043940](https://doi.org/10.1109/IRDS.2002.1043940).
- [12] Asoh H, et al. Jijo-2: an office robot that communicates and learns. *IEEE Intelligent Systems* 2001;16(5):46–55, [doi:10.1109/5254.956081](https://doi.org/10.1109/5254.956081).
- [13] Savage-Carmona J, Billinghamurst M, Holden A. The VirBot: a virtual reality robot driven with multimodal commands. *Expert Systems with Applications* 1998;15(3–4):413–9, [doi:10.1016/S0957-4174\(98\)00037-2](https://doi.org/10.1016/S0957-4174(98)00037-2).
- [14] Ayres T, Nolan B. Voice activated command and control with speech recognition over WiFi. *Science of Computer Programming* 2006;59(1–2):109–26, [doi:10.1016/j.scico.2005.07.007](https://doi.org/10.1016/j.scico.2005.07.007).
- [15] Savage J, Llarena A, Carrera G, Cuellar S, Esparza D, Minami Y, Penuelas U. VirBot: a system for the operation of mobile robots. *Lecture Notes in Computer Science* 2008;5001:512–9, [doi:10.1007/978-3-540-68847-1_55](https://doi.org/10.1007/978-3-540-68847-1_55).
- [16] Wasfy A, Wasfy T, Noor A. Intelligent virtual environment for process training. *Advances in Engineering Software* 2004;35(6):227–355, [doi:10.1016/j.advengsoft.2004.04.005](https://doi.org/10.1016/j.advengsoft.2004.04.005).
- [17] Pires J. Robot-by-voice: experiments on commanding an industrial robot using the human voice. *Industrial Robot* 2005;32(6):505–11, [doi:10.1108/01439910510629244](https://doi.org/10.1108/01439910510629244).
- [18] Wang J, Su J, Xi Y. COM-based software architecture for multisensory fusion system. *Inf. Fusion* 2001;2(4):261–70, [doi:10.1016/S1566-2535\(01\)00042-2](https://doi.org/10.1016/S1566-2535(01)00042-2).
- [19] Pires J. Industrial robots programming: building applications for the factories of the future. New York, USA: Springer; 2006.
- [20] Veiga G, Pires J, Nilsson K. Experiments with service-oriented architectures for industrial robotic cells programming. *Robotics and Computer-Integrated Manufacturing* 2009;25(4–5):746–55, [doi:10.1016/j.rcim.2008.09.001](https://doi.org/10.1016/j.rcim.2008.09.001).
- [21] Schlick C, Daude R, Luczak H, Weck M, Springer J. Head-mounted display for supervisory control in autonomous production cells. *Elsevier Displays* 1997;17(3–4):199–206, [doi:10.1016/S0141-9382\(96\)01036-0](https://doi.org/10.1016/S0141-9382(96)01036-0).
- [22] Lauria S. Talking to machines: introducing robot perception to resolve speech recognition uncertainties. *Circuits Systems and Signal Processing* 2007;26(4):513–26, [doi:10.1007/s00034-007-4005-9](https://doi.org/10.1007/s00034-007-4005-9).
- [23] Jelinek E. Statistical methods for speech recognition. Cambridge, MA: MIT Press; 1997.
- [24] Rosenfeld R. Two decades of statistical language modeling: where do we go from here? *Proceedings of the IEEE* 2000;88(8):1270–8, [doi:10.1109/5.880083](https://doi.org/10.1109/5.880083).
- [25] Emami A, Jelinek F. A neural syntactic language model. *Machine Learning* 2005;60(1–3):195–227, [doi:10.1007/s10994-005-0916-y](https://doi.org/10.1007/s10994-005-0916-y).
- [26] Gildea D, Jurafsky D. Automatic labeling of semantic roles. *Computational Linguistics* 2002;28(3):245–88, [doi:10.1162/089120102760275983](https://doi.org/10.1162/089120102760275983).
- [27] Paek T, Maxwell D. Improving command and control speech recognition on mobile devices: using predictive user models for language modeling. *User Modeling and User-Adapted Interaction* 2007;17:93–117, [doi:10.1007/s11257-006-9021-6](https://doi.org/10.1007/s11257-006-9021-6).
- [28] Bosch A, Daelemans W. Improving sequence segmentation learning by predicting trigrams. In: Proceedings of the 9th conference on computational natural language learning, Ann Arbor, MI; 2005. p. 80–7.
- [29] Sun J, Karray F, Basir O, Kamel M. Natural language understanding through fuzzy logic inference and its application to speech recognition. In: Proceedings of the IEEE international conference on fuzzy systems, Honolulu, HI; 2002. p. 1120–5, [doi:10.1109/FUZZ.2002.1006661](https://doi.org/10.1109/FUZZ.2002.1006661).

- [30] Lepratti R. Advanced human-machine system for intelligent manufacturing. *Journal of Intelligent Manufacturing* 2006;17(6):653–66, [doi:10.1007/s10845-006-0035-z](https://doi.org/10.1007/s10845-006-0035-z).
- [31] Ge R, Mooney R. A statistical semantic parser that integrates syntax and semantics. In: *Proceedings of the 9th conference on computational natural language learning*, Ann Arbor, MI; 2005. p. 9–16.
- [32] Schank R. A conceptual dependency parser for natural language. In: *Proceedings of the 1969 conference on computational linguistics*, Stroudsburg, PA; 1969. p. 1–28. [doi:10.3115/990403.990405](https://doi.org/10.3115/990403.990405).
- [33] Kulyukin V, Steele A. Input recognition in voice control interfaces to three-tier autonomous robots. In: *Proceedings of the international LISP conference*, San Francisco, CA; 2002. p. 1338–43.
- [34] Kulyukin V, Steele A. Instruction and action in the three-tiered robot architecture. In: *Proceedings of the international symposium on robotics and automation*, Toluca, Mexico; 2002.
- [35] Rogowski A. Robotized cell remote control using voice commands in natural language. In: *Proceedings of the 15th international conference on methods and models in automation and robotics*, Miedzyzdroje, Poland; 2010. p. 383–6.